Mini-Projet Cloud Computing

Plateforme de Microservices avec Docker

Réalisé par :

Aymen Settey

2ème année — Filière INLOG 2

Module :	Cloud Computing
Année universitaire :	2024–2025
Encadrant :	*[Nom de l'encadrant]*

28 mars 2026

Table des matières

1 Architecture Générale **1**
1.1 Vue d'ensemble 1
1.2 Flux de données 1

2 Conteneurs **2**
2.1 Flask App (image custom) 2
2.2 Healthchecks 2
2.3 Scalabilité horizontale 3

3 Réseau **3**
3.1 Réseau Docker 3
3.2 Ports exposés 4
3.3 Reverse Proxy — Nginx 4

4 Volumes **5**
4.1 Bind mounts (lecture seule) 5

5 Extensions **5**
5.1 Monitoring — Prometheus & Grafana 5
5.2 Cache Redis 6
5.3 CI/CD — GitHub Actions 7
5.4 Base de données — PostgreSQL 7

Table des figures

1 Diagramme d'architecture de la plateforme 1
2 État des conteneurs avec `docker compose ps` 2
3 Test de scalabilité — hostnames différents confirmant la répartition de charge 3
4 Réponse de `GET /tasks` — liste des tâches 4
5 Interface Prometheus — requête `http_requests_total` 6
6 Interface Grafana — tableau de bord avec la source Prometheus 6
7 Pipeline CI/CD GitHub Actions — exécution réussie 7

Architecture Générale

Ce projet implémente une architecture microservices complète déployée via **Docker Compose**. L'application est une API REST de gestion de tâches (TODO) construite avec Flask, accompagnée de services de support pour le stockage, le cache, le reverse proxy et le monitoring.

Vue d'ensemble

L'architecture suit le modèle suivant :

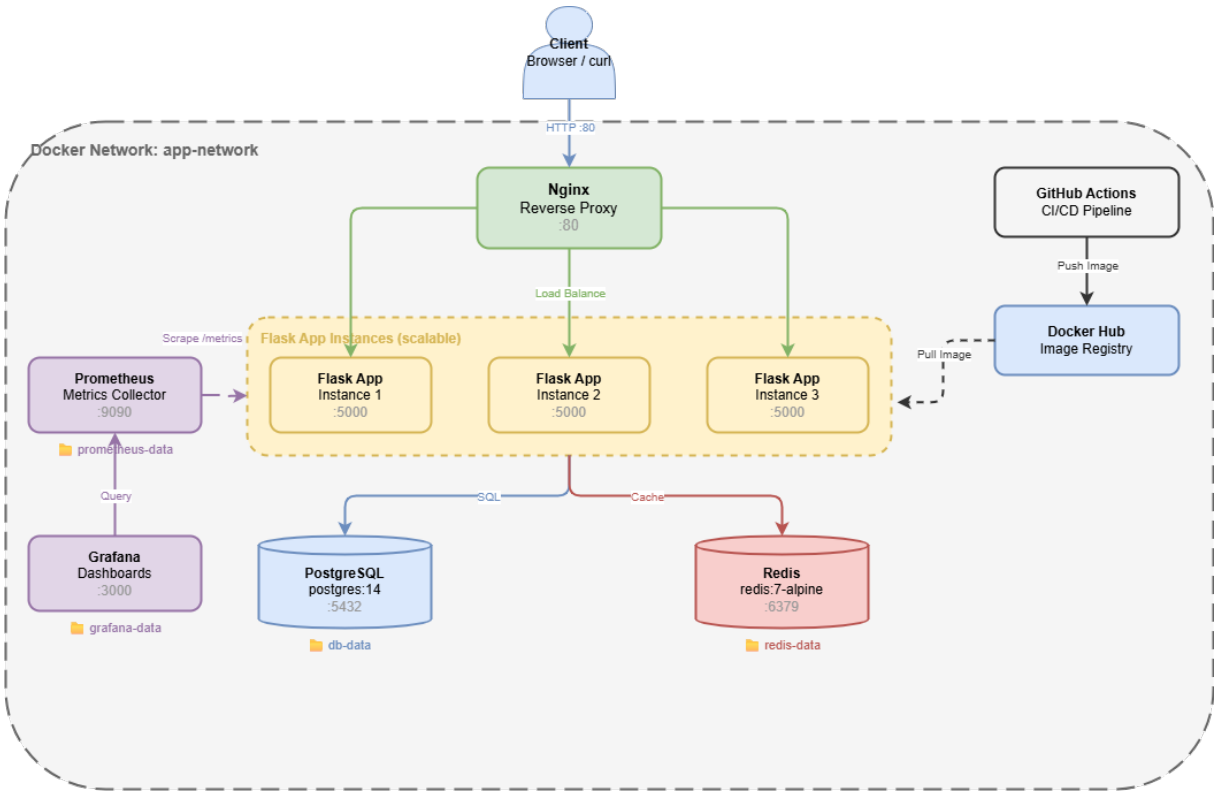

FIGURE 1 – Diagramme d'architecture de la plateforme

Flux de données

1. Le **client** envoie une requête HTTP sur le port 80.
2. **Nginx** reçoit la requête et la distribue vers l'une des instances Flask via un algorithme de répartition de charge.
3. L'instance **Flask** traite la requête :
   — Pour les lectures (`GET /tasks`) : vérifie d'abord le cache **Redis** (TTL 30s), sinon interroge **PostgreSQL**.
   — Pour les écritures (`POST`, `PUT`, `DELETE`) : modifie la base PostgreSQL et invalide le cache Redis.
4. **Prometheus** scrape les métriques de chaque instance Flask toutes les 15 secondes.
5. **Grafana** interroge Prometheus pour afficher les tableaux de bord.

Conteneurs

Le projet utilise **6 services** conteneurisés. Le tableau ci-dessous résume chaque conteneur :

Service	Image	Port	Rôle
`nginx`	`nginx:alpine`	80	Reverse proxy & load balancer
`app`	Image custom (Flask)	5000	API REST TODO
`db`	`postgres:14`	5432	Base de données relationnelle
`redis`	`redis:7-alpine`	6379	Cache & compteur de visites
`prometheus`	`prom/prometheus`	9090	Collecte de métriques
`grafana`	`grafana/grafana`	3000	Visualisation des métriques

Table 1 – Liste des conteneurs et leurs rôles

```
PS D:\coding Projects\cloud_project> docker compose up -d --build
[+] Running 54/55
 ✓ db Pulled
      830.0s
 ✓ grafana Pulled
      939.1s
 ✓ redis Pulled
      261.0s
 ✓ nginx Pulled
      295.9s
 ✓ prometheus Pulled
      721.4s
```

Figure 2 – État des conteneurs avec `docker compose ps`

Flask App (image custom)

L'image est construite à partir d'un `Dockerfile` basé sur `python:3.11-slim`. L'application est servie par **Gunicorn** avec 2 workers. Les dépendances principales sont :

- `Flask` + `Flask-SQLAlchemy` — framework web et ORM
- `psycopg2-binary` — driver PostgreSQL
- `redis` — client Redis pour le cache
- `prometheus_client` — exposition des métriques

Healthchecks

Chaque service critique dispose d'un **healthcheck** Docker :

- `db` : `pg_isready -U admin -d tasks`
- `redis` : `redis-cli ping`

— `app` : `curl -f http://localhost:5000/health`

L'option `depends_on` avec `condition: service_healthy` garantit l'ordre de démarrage : la base de données et Redis doivent être prêts avant le lancement de Flask.

Scalabilité horizontale

L'application Flask peut être mise à l'échelle horizontalement :

```
docker compose up -d --scale app=3
```

Nginx découvre automatiquement les nouvelles instances grâce au résolveur DNS interne de Docker (`127.0.0.11`) et distribue le trafic entre elles.

```
PS D:\coding Projects\cloud_project> docker compose up -d --scale app=3
 [+] Running 8/8
  ✓ Container postgres            Healthy
          0.7s
  ✓ Container prometheus          Running
          0.0s
  ✓ Container redis               Healthy
          0.7s
  ✓ Container cloud_project-app-1 Running
          0.0s
  ✓ Container grafana             Running
          0.0s
  ✓ Container cloud_project-app-3 Started
          1.1s
  ✓ Container cloud_project-app-2 Started
          0.9s
  ✓ Container nginx               Running
          0.0s
```

FIGURE 3 – Test de scalabilité — hostnames différents confirmant la répartition de charge

Réseau

Réseau Docker

Tous les services sont connectés à un réseau bridge unique nommé `app-network` :

```
networks:
  app-network:
    driver: bridge
```

Ce réseau permet la **résolution DNS automatique** : chaque service peut communiquer avec les autres en utilisant le nom du service comme hostname (ex. : `db`, `redis`, `app`).

Ports exposés

Service	Port hôte	Port conteneur
Nginx	80	80
PostgreSQL	5432	5432
Redis	6379	6379
Prometheus	9090	9090
Grafana	3000	3000

TABLE 2 – Mapping des ports

Note : Les instances Flask ne publient pas de port sur l'hôte — elles ne sont accessibles que via Nginx à travers le réseau interne Docker.

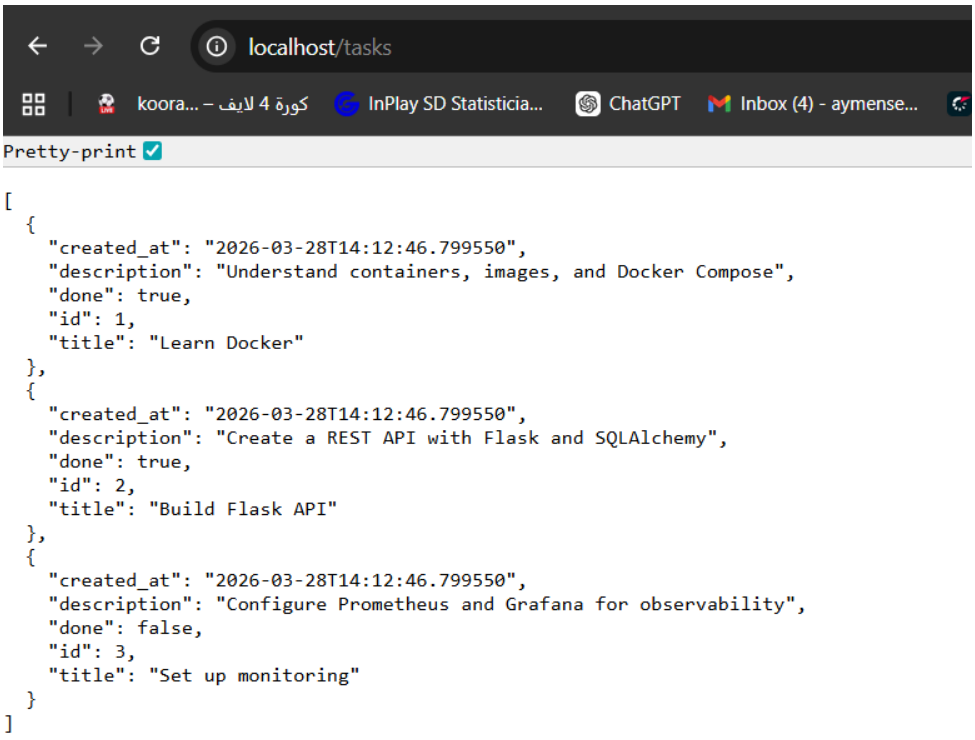

FIGURE 4 – Réponse de `GET /tasks` — liste des tâches

Reverse Proxy — Nginx

Nginx est configuré comme point d'entrée unique avec les fonctionnalités suivantes :

— **Répartition de charge** : résolution DNS dynamique via `resolver 127.0.0.11`

— **Rate limiting** : 10 requêtes/seconde par IP, avec un burst de 20

— **Compression Gzip** : activée pour JSON, JavaScript et CSS

— **Headers proxy** : `X-Real-IP`, `X-Forwarded-For`, `X-Forwarded-Proto`

Volumes

Les données persistantes sont stockées dans des **volumes nommés Docker**, garantissant la survie des données au redémarrage des conteneurs.

Volume	Monté dans	Usage
`db-data`	`/var/lib/postgresql/data`	Données PostgreSQL
`redis-data`	`/data`	Données Redis (RDB)
`prometheus-data`	`/prometheus`	Métriques Prometheus (TSDB)
`grafana-data`	`/var/lib/grafana`	Dashboards & config Grafana

Table 3 – Volumes nommés et leurs usages

Bind mounts (lecture seule)

Les fichiers de configuration sont montés en **lecture seule** (`:ro`) pour éviter toute modification accidentelle par les conteneurs :

— `./db/init.sql` → `/docker-entrypoint-initdb.d/init.sql:ro`

— `./nginx/nginx.conf` → `/etc/nginx/conf.d/default.conf:ro`

— `./prometheus/prometheus.yml` → `/etc/prometheus/prometheus.yml:ro`

— `./grafana/provisioning/` → `/etc/grafana/provisioning/:ro`

Extensions

Monitoring — Prometheus & Grafana

L'application Flask expose un endpoint `/metrics` au format Prometheus avec les métriques suivantes :

— `http_requests_total` (Counter) — nombre total de requêtes, avec labels : `method`, `endpoint`, `status`

— `http_request_duration_seconds` (Histogram) — temps de réponse des requêtes

— `visit_count_total` (Counter) — nombre total de visites sur la page d'accueil

Prometheus utilise la **découverte DNS** (`dns_sd_configs`) pour scraper automatiquement toutes les instances Flask, même après un scaling dynamique.

Grafana est pré-configuré avec Prometheus comme source de données grâce au **provisioning automatique** (fichier YAML dans `provisioning/datasources/`).

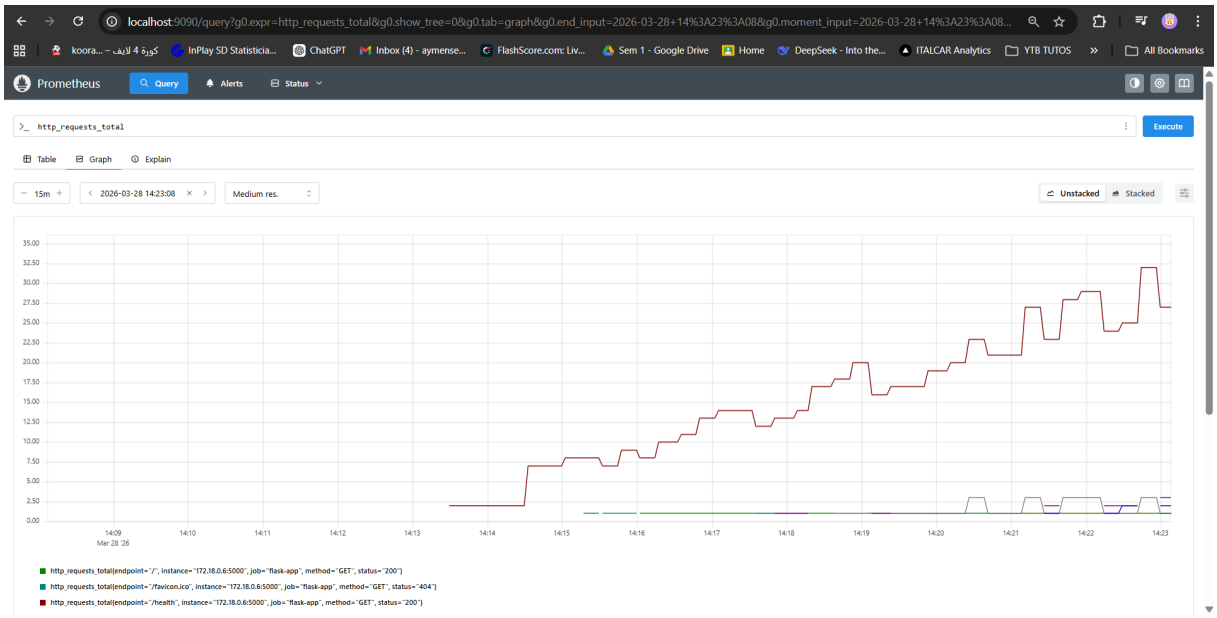

FIGURE 5 – Interface Prometheus — requête `http_requests_total`

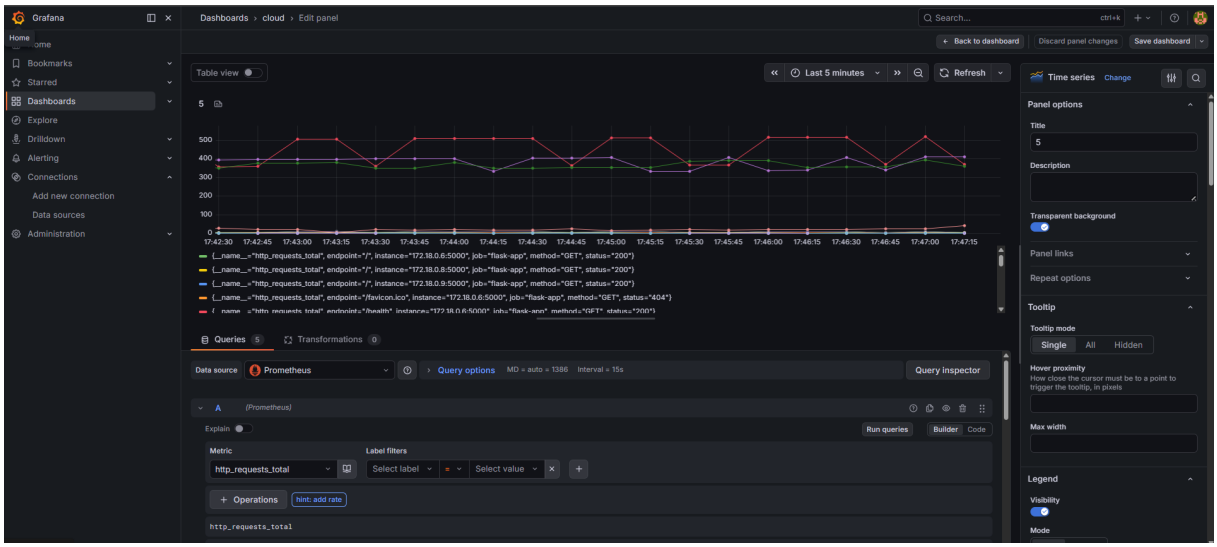

FIGURE 6 – Interface Grafana — tableau de bord avec la source Prometheus

Cache Redis

Redis est utilisé pour deux fonctions :

1. **Cache des requêtes** : les réponses de `GET /tasks` sont mises en cache avec un TTL de 30 secondes. Toute opération d'écriture (`POST`, `PUT`, `DELETE`) invalide le cache.
2. **Compteur atomique de visites** : utilisation de `INCR` pour compter les visites sur `GET /` de manière thread-safe et partagée entre toutes les instances.

CI/CD — GitHub Actions

Le pipeline CI/CD est déclenché automatiquement sur chaque `push` ou `pull request` vers la branche `main`. Les étapes sont :

1. **Build** de l'image Docker Flask
2. **Push** vers Docker Hub (uniquement sur `push`, pas sur les PR)
3. **Démarrage** de tous les services via `docker compose up`
4. **Tests d'intégration** : vérification des endpoints `/`, `/tasks`, `/metrics`
5. **Nettoyage** : arrêt et suppression des conteneurs

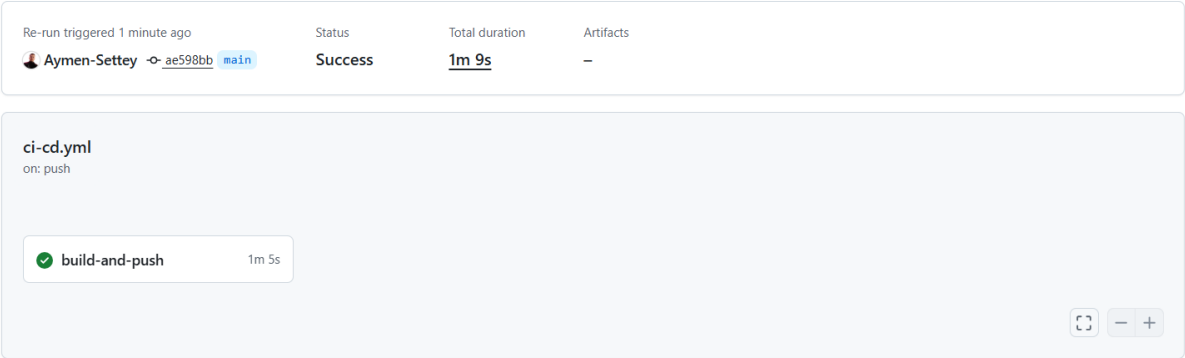

FIGURE 7 – Pipeline CI/CD GitHub Actions — exécution réussie

Base de données — PostgreSQL

PostgreSQL est initialisé avec un script SQL (`init.sql`) qui :

— Crée la table `tasks` avec les colonnes : `id`, `title`, `description`, `done`, `created_at`

— Insère 3 tâches d'exemple pour faciliter les tests

Flask utilise également `db.create_all()` au démarrage pour garantir l'existence de la table, indépendamment du script d'initialisation.